

Data analytics Capstone project

Exploratory Data Analysis Of Netflix Content



AIM

The aim of this project is to analyze Spotify music data to understand song characteristics, listener preferences, genre trends, and popularity patterns over time, and to build predictive models that can forecast song popularity using audio features. The project leverages data analytics and visualization techniques to derive meaningful insights from large-scale Spotify datasets.

OBJECTIVE

To explore and understand Spotify datasets containing track-level, artist-level, genre-level, and year-wise information.

To perform Exploratory Data Analysis (EDA) to study the distribution of audio features such as danceability, energy, tempo, valence, and loudness.

To analyze trends over time, including how song popularity and audio features have evolved across different years.

To identify popular genres and user listening preferences based on average popularity and musical attributes.

To examine relationships and correlations between different audio features to understand how they influence each other.

To visualize key insights using interactive Plotly charts such as histograms, line plots, bar charts, scatter plots, and heatmaps.

To build predictive models that forecast song popularity using Spotify audio features.

PURPOSE

The purpose of this project is to demonstrate how data analytics can be applied to the music streaming domain to extract actionable insights from large datasets. By analyzing Spotify audio features and popularity metrics, the project aims to:

Help understand what makes a song popular

Identify changing music trends and listener preferences

Show how data-driven decision-making can support artists, producers, and streaming platforms

Apply predictive analytics to forecast future outcomes such as song popularity

This project also serves as a practical learning experience in data analytics, covering data preprocessing, visualization, correlation analysis, and predictive modeling using real-world datasets.

Tools



Python

Use: A high-level programming language widely used for data analysis, automation, and AI.

Features:

Easy to learn and read.

Huge library support for data, web, and AI.

Backbone for libraries like Pandas, NumPy, Matplotlib, and



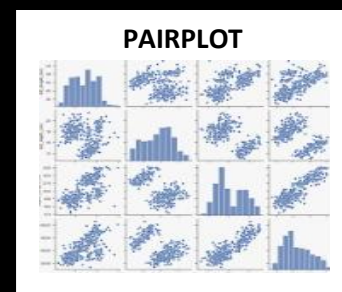
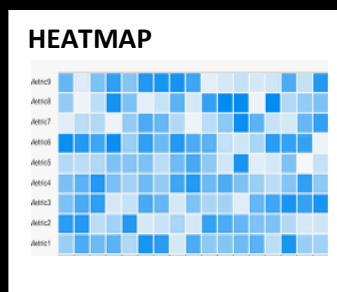
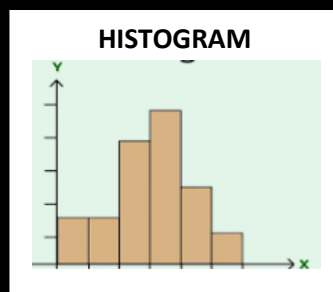
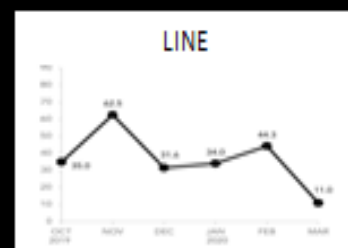
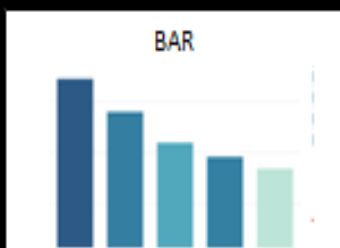
Plotly

Plotly is an interactive data visualization library used to create dynamic and engaging charts.

It supports various visualizations such as bar charts, line charts, scatter plots, pie charts, and maps.

Plotly allows users to interact with data through features like zooming, hovering, and filtering.

Graphs



PROBLEM STATEMENTS

Task 1: Load the dataset into the Jupyter notebook.

Task 2: Inspect and clean the data (handle missing values, duplicates, etc.).

Task 3: Perform exploratory data analysis (EDA) to understand the basic characteristics of the data.

Task 4: Analyze the distribution of various features (e.g., danceability, energy, tempo).

Task 5: Identify trends over time (e.g., popularity of genres, changes in song features).

Task 6: Examine correlations between different features (e.g., energy vs. danceability).

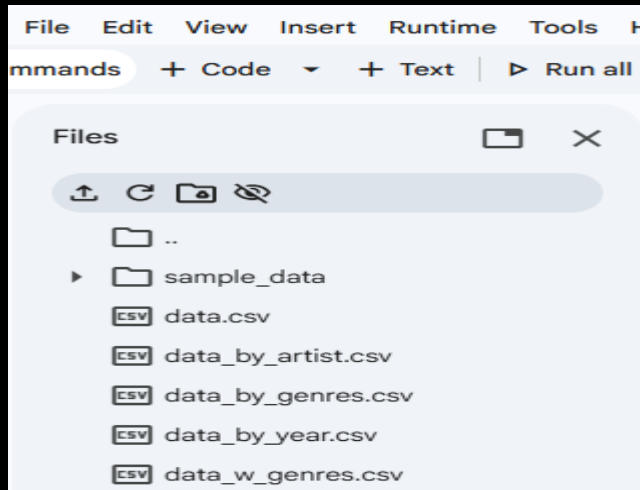
Task 7: Analyze user preferences and listening habits.

Task 8: Create visualizations to represent key findings (e.g., bar charts, line graphs, scatter plots).

Task 9: Use advanced visualization techniques (e.g., heatmaps, pair plots) to uncover deeper insights.

DATA OVERVIEW

Data Imported :



Overview :-

Here we imported/ load the 5 CSV / Excel file on which we are going to analysis the data.

Import python libraries :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
```

Overview :-

To perform analysis we have to import few libraries of python I.e: Pandas (for data cleaning and manipulatio), Matplotlib and seaborn (for Visualisation), & Numpy (for performing numerical operations).

READ FIRST CSV FILE

```
df_tracks = pd.read_csv("/content/data.csv")  
df_tracks
```

	valence	year	acousticness	artists	danceability	duration_ms	energy
0	0.0594	1921	0.98200	['Sergei Rachmaninoff', 'James Levine', 'Berli...	0.279	831667	0.211
1	0.9630	1921	0.73200	['Dennis Day']	0.819	180533	0.341
2	0.0394	1921	0.96100	['KHP Kridhamardawa Karaton Ngayogyakarta Hadi...	0.328	500062	0.166
3	0.1650	1921	0.96700	['Frank Parker']	0.275	210000	0.309

170649	0.7340	2020	0.20600	['Ashnikko']	0.717	150654	0.753
170650	0.6370	2020	0.10100	['MAMAMOO']	0.634	211280	0.858
170651	0.1950	2020	0.00998	['Eminem']	0.671	337147	0.623
170652	0.6420	2020	0.13200	['KEVVO', 'J Balvin']	0.856	189507	0.721

170653 rows x 19 columns

OVERVIEW :-

Using pandas library ,we read the first excel file . Which contains 170653 rows and 19 columns.

CHECK DATA SHAPE & DESCRIBE THE NUMERICAL VALUES OF DATASET

df_tracks.shape							
(170653, 19)							
df_tracks.info()							
*** <class 'pandas.core.frame.DataFrame'> RangeIndex: 170653 entries, 0 to 170652 Data columns (total 19 columns): # Column Non-Null Count Dtype --- 0 valence 170653 non-null float64 1 year 170653 non-null int64 2 acousticness 170653 non-null float64 3 artists 170653 non-null object 4 danceability 170653 non-null float64 5 duration_ms 170653 non-null int64 6 energy 170653 non-null float64 7 explicit 170653 non-null int64 8 id 170653 non-null object 9 instrumentalness 170653 non-null float64 10 key 170653 non-null int64 11 liveness 170653 non-null float64 12 loudness 170653 non-null float64 13 mode 170653 non-null int64 14 name 170653 non-null object 15 popularity 170653 non-null int64 16 release_date 170653 non-null object 17 speechiness 170653 non-null float64 18 tempo 170653 non-null float64							
df_tracks.describe()							
	valence	year	acousticness	danceability	duration_ms	energy	
count	170653.000000	170653.000000	170653.000000	170653.000000	1.706530e+05	170653.000000	17
mean	0.528587	1976.787241	0.502115	0.537396	2.309483e+05	0.482389	
std	0.263171	25.917853	0.376032	0.176138	1.261184e+05	0.267646	
min	0.000000	1921.000000	0.000000	0.000000	5.108000e+03	0.000000	
25%	0.317000	1956.000000	0.102000	0.415000	1.698270e+05	0.255000	
50%	0.540000	1977.000000	0.516000	0.548000	2.074670e+05	0.471000	
75%	0.747000	1999.000000	0.893000	0.668000	2.624000e+05	0.703000	
max	1.000000	2020.000000	0.996000	0.988000	5.403500e+06	1.000000	

OVERVIEW:-

This Dataframe is confined to have 170653 rows and 19 column and using df.describe() we found that there are 15 numerical column.

READ SECOND CSV FILE



```
df_artist = pd.read_csv("/content/data_by_artist.csv")  
df_artist
```

...	mode	count	acousticness	artists	danceability	duration_ms	energy
0	1	9	0.590111	"Cats" 1981 Original London Cast	0.467222	250318.555556	0.394003
1	1	26	0.862538	"Cats" 1983 Broadway Cast	0.441731	287280.000000	0.406808
2	1	7	0.856571	"Fiddler On The Roof" Motion Picture Chorus	0.348286	328920.000000	0.286571
3	1	27	0.884926	"Fiddler On The Roof" Motion Picture	0.425074	262890.962963	0.245770

28675	1	2	0.512000	麥志誠	0.356000	198773.000000	0.306000
28676	0	2	0.541000	黃品源	0.578000	293840.000000	0.334000
28677	1	11	0.785455	黃國隆	0.570818	174582.727273	0.148400
28678	1	2	0.381000	黑豹	0.353000	316160.000000	0.686000
28679	1	2	0.568000	조정현	0.447000	237688.000000	0.215000

28680 rows x 15 columns

OVERVIEW :-

Using pandas library ,we read the first excel file . Which contains 28680 rows and 15 columns.

CHECK DATA SHAPE & DESCRIBE THE NUMERICAL VALUES OF DATASET

```
df_artist.shape
```

```
(28680, 15)
```



```
df_artist.info()
```

```
... <class 'pandas.core.frame.DataFrame'>
RangeIndex: 28680 entries, 0 to 28679
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   mode                  28680 non-null  int64
1   count                 28680 non-null  int64
2   acousticness          28680 non-null  float64
3   artists               28680 non-null  object
4   danceability           28680 non-null  float64
5   duration_ms           28680 non-null  float64
6   energy                 28680 non-null  float64
7   instrumentalness       28680 non-null  float64
8   liveness               28680 non-null  float64
9   loudness               28680 non-null  float64
10  speechiness           28680 non-null  float64
11  tempo                  28680 non-null  float64
12  valence                28680 non-null  float64
13  popularity             28680 non-null  float64
14  key                    28680 non-null  int64
dtypes: float64(11), int64(3), object(1)
memory usage: 3.3+ MB
```

```
df_artist.describe()
```

	mode	count	acousticness	danceability	duration_ms	energy
count	28680.000000	28680.000000	28680.000000	28680.000000	2.868000e+04	28680.000000
mean	0.759170	13.847211	0.498373	0.546490	2.388780e+05	0.497488
std	0.427595	53.372544	0.370614	0.176474	1.211318e+05	0.254885
min	0.000000	1.000000	0.000000	0.000000	1.879550e+04	0.000000
25%	1.000000	2.000000	0.122296	0.431000	1.823304e+05	0.283568
50%	1.000000	3.000000	0.478458	0.557000	2.186400e+05	0.504000
75%	1.000000	8.000000	0.896000	0.675000	2.684670e+05	0.702783
max	1.000000	3169.000000	0.996000	0.986000	5.403500e+06	1.000000

OVERVIEW :-

This Dataframe is confined to have 28680 rows and 15 column and using `df.describe()` we found that there are 14 numerical column.

READ THIRD CSV FILE

```
df_genres = pd.read_csv("/content/data_by_genres.csv")  
df_genres
```

...	mode	genres	acousticness	danceability	duration_ms	energy	instrumentalness
0	1	21st century classical	0.979333	0.162883	1.602977e+05	0.071317	0.606834
1	1	432hz	0.494780	0.299333	1.048887e+06	0.450678	0.477762
2	1	8-bit	0.762000	0.712000	1.151770e+05	0.818000	0.876000
3	1	[]	0.651417	0.529093	2.328809e+05	0.419146	0.205309
4	1	a cappella	0.676557	0.538961	1.906285e+05	0.316434	0.003003

...	...	...	...	...	...	...	...
2968	1	zolo	0.222625	0.547082	2.580991e+05	0.610240	0.143872
2969	0	zouglou	0.161000	0.863000	2.063200e+05	0.909000	0.000000
2970	1	zouk	0.263261	0.748889	3.060728e+05	0.622444	0.257227
2971	0	zurich indie	0.993000	0.705667	1.984173e+05	0.172667	0.468633
2972	1	zydeco	0.421038	0.629409	1.716717e+05	0.609369	0.019248

2973 rows x 14 columns

OVERVIEW :-

Using pandas library ,we read the first excel file . Which contains 2973 rows and 14 columns.

CHECK DATA SHAPE & DESCRIBE THE NUMERICAL VALUES OF DATASET

```
df_genres.shape
```

```
(2973, 14)
```

```
df_genres.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2973 entries, 0 to 2972
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   mode                  2973 non-null  int64  
 1   genres                2973 non-null  object  
 2   acousticness          2973 non-null  float64 
 3   danceability          2973 non-null  float64 
 4   duration_ms           2973 non-null  float64 
 5   energy                2973 non-null  float64 
 6   instrumentalness       2973 non-null  float64 
 7   liveness              2973 non-null  float64 
 8   loudness              2973 non-null  float64 
 9   speechiness           2973 non-null  float64 
10   tempo                 2973 non-null  float64 
11   valence               2973 non-null  float64 
12   popularity            2973 non-null  float64 
13   key                   2973 non-null  int64  
dtypes: float64(11), int64(2), object(1)
memory usage: 325.3+ KB
```

```
df_genres.describe()
```

	mode	acousticness	danceability	duration_ms	energy	instrumentalness
count	2973.000000	2973.000000	2973.000000	2.973000e+03	2973.000000	2973.000000
mean	0.833165	0.401241	0.537187	2.517209e+05	0.561143	0.211366
std	0.372891	0.319760	0.150668	9.465686e+04	0.234486	0.267329
min	0.000000	0.000003	0.056900	3.094600e+04	0.001002	0.000000
25%	1.000000	0.119050	0.441202	2.063788e+05	0.395058	0.004835
50%	1.000000	0.321745	0.546496	2.375453e+05	0.601195	0.080700
75%	1.000000	0.673991	0.647500	2.772720e+05	0.730127	0.343333
max	1.000000	0.996000	0.929000	2.382587e+06	0.994667	0.992000

OVERVIEW :-

This Dataframe is confined to have 2973 rows and 14 column and using `df.describe()` we found that there are 13 numerical column.

READ FOURTH CSV FILE



```
df_year = pd.read_csv("/content/data_by_year.csv")  
df_year
```

...	mode	year	acousticness	danceability	duration_ms	energy	instrumentalness
0	1	1921	0.886896	0.418597	260537.166667	0.231815	0.344878
1	1	1922	0.938592	0.482042	165469.746479	0.237815	0.434195
2	1	1923	0.957247	0.577341	177942.362162	0.262406	0.371733
3	1	1924	0.940200	0.549894	191046.707627	0.344347	0.581701
4	1	1925	0.962607	0.573863	184986.924460	0.278594	0.418297
...	...	...	...	...	...	...	...
95	1	2016	0.284171	0.600202	221396.510295	0.592855	0.093984
96	1	2017	0.286099	0.612217	211115.696787	0.590421	0.097091
97	1	2018	0.267633	0.663500	206001.007133	0.602435	0.054217
98	1	2019	0.278299	0.644814	201024.788096	0.593224	0.077640
99	1	2020	0.219931	0.692904	193728.397537	0.631232	0.016376

100 rows x 14 columns

OVERVIEW :-

Using pandas library ,we read the first excel file . Which contains 100 rows and 14 columns.

CHECK DATA SHAPE & DESCRIBE THE NUMERICAL VALUES OF DATASET

```
df_year.shape
```

```
(100, 14)
```

```
df_year.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   mode                  100 non-null   int64
1   year                  100 non-null   int64
2   acousticness          100 non-null   float64
3   danceability           100 non-null   float64
4   duration_ms           100 non-null   float64
5   energy                 100 non-null   float64
6   instrumentalness       100 non-null   float64
7   liveness               100 non-null   float64
8   loudness               100 non-null   float64
9   speechiness            100 non-null   float64
10  tempo                  100 non-null   float64
```

```
df_year.describe()
```

	mode	year	acousticness	danceability	duration_ms	energy
count	100.0	100.000000	100.000000	100.000000	100.000000	100.000000
mean	1.0	1970.500000	0.556317	0.536783	227296.752234	0.452705
std	0.0	29.011492	0.275358	0.052356	25630.048065	0.161738
min	1.0	1921.000000	0.219931	0.414445	156881.657475	0.207948
25%	1.0	1945.750000	0.289516	0.500800	210889.193536	0.280733
50%	1.0	1970.500000	0.459190	0.540976	235520.850833	0.495997
75%	1.0	1995.250000	0.856711	0.570948	247702.738058	0.598008
max	1.0	2020.000000	0.962607	0.692904	267677.823086	0.681778

OVERVIEW :-

This Dataframe is confined to have 100 rows and 14 column and using `df.describe()` we found that there are 14 numerical column.

READ FIFTH CSV FILE

df_tracks_genres							
...							
	genres	artists	acousticness	danceability	duration_ms	energy	id
0	['show tunes']	"Cats" 1981 Original London Cast	0.590111	0.467222	250318.555556	0.394003	
1	[]	"Cats" 1983 Broadway Cast	0.862538	0.441731	287280.000000	0.406808	
2	[]	"Fiddler On The Roof" Motion Picture Chorus	0.856571	0.348286	328920.000000	0.286571	
3	[]	"Fiddler On The Roof" Motion Picture Orchestra	0.884926	0.425074	262890.962963	0.245770	

28675	[]	麥志誠	0.512000	0.356000	198773.000000	0.306000	
28676	['c-pop', 'classic cantopop', 'classic mandopo...]	黃品源	0.541000	0.578000	293840.000000	0.334000	
28677	[]	黃國隆	0.785455	0.570818	174582.727273	0.148400	
28678	['chinese indie', 'chinese indie rock']	黑豹	0.381000	0.353000	316160.000000	0.686000	
28679	['classic korean pop']	조정현	0.568000	0.447000	237688.000000	0.215000	

28680 rows x 16 columns

OVERVIEW :-

Using pandas library ,we read the first excel file . Which contains 28680 rows and 16 columns.

CHECK DATA SHAPE & DESCRIBE THE NUMERICAL VALUES OF DATASET



```
df_tracks_genres.shape
```

```
... (28680, 16)
```

```
df_tracks_genres.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28680 entries, 0 to 28679
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  ---                ---
0   genres                 28680 non-null  object
1   artists                28680 non-null  object
2   acousticness           28680 non-null  float64
3   danceability           28680 non-null  float64
4   duration_ms            28680 non-null  float64
5   energy                 28680 non-null  float64
6   instrumentalness        28680 non-null  float64
7   liveness               28680 non-null  float64
8   loudness               28680 non-null  float64
9   speechiness            28680 non-null  float64
10  tempo                  28680 non-null  float64
11  valence                28680 non-null  float64
12  popularity             28680 non-null  float64
13  key                    28680 non-null  int64
14  mode                   28680 non-null  int64
15  count                  28680 non-null  int64
dtypes: float64(11), int64(3), object(2)
memory usage: 3.5+ MB
```

```
df_tracks_genres.describe()
```

	acousticness	danceability	duration_ms	energy	instrumentalness
count	28680.000000	28680.000000	2.868000e+04	28680.000000	28680.000000
mean	0.498373	0.546490	2.388780e+05	0.497488	0.174756
std	0.370614	0.176474	1.211318e+05	0.254885	0.298406
min	0.000000	0.000000	1.879550e+04	0.000000	0.000000
25%	0.122296	0.431000	1.823304e+05	0.283568	0.000004
50%	0.478458	0.557000	2.186400e+05	0.504000	0.001880
75%	0.896000	0.675000	2.684670e+05	0.702783	0.215291
max	0.996000	0.986000	5.403500e+06	1.000000	1.000000

OVERVIEW :-

This Dataframe is confined to have 28680 rows and 14 column and using `df.describe()` we found that there are 14 numerical column.

CHECKING NULL VALUES

df_tracks.isnull().sum()

...

0

valence0

year0

acousticness0

artists0

danceability0

duration_ms0

energy0

explicit0

id0

instrumentalness0

df_artist.isnull().sum()

...

0

mode0

count0

acousticness0

artists0

danceability0

duration_ms0

energy0

instrumentalness0

liveness0

loudness0

df_genres.isnull().sum()

0

mode0

genres0

acousticness0

danceability0

duration_ms0

energy0

instrumentalness0

liveness0

loudness0

speechiness0

tempo0

valence0

popularity0

df_year.isnull().sum()

...

0

mode0

year0

acousticness0

danceability0

duration_ms0

energy0

instrumentalness0

liveness0

loudness0

speechiness0

tempo0

valence0

df_tracks_genres.isnull().sum()

...

0

genres0

artists0

acousticness0

danceability0

duration_ms0

energy0

instrumentalness0

liveness0

loudness0

speechiness0

tempo0

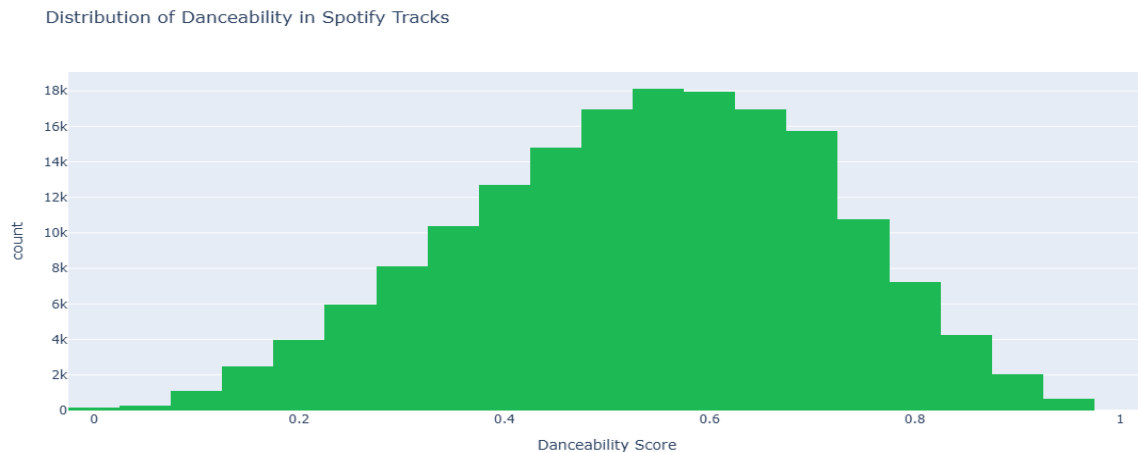
OVERVIEW:

Our data frames does not contain any null values , all columns have been filled properly already .

INSIGHTS

(A) Distribution of Danceability in Spotify Tracks

```
# Task 4: Analyze the distribution of various features (e.g., danceability, energy, tempo).
import plotly.express as px
px.histogram(
    df_tracks,
    x="danceability",
    nbins=30,
    title="Distribution of Danceability in Spotify Tracks",
    labels={"danceability": "Danceability Score"},
    color_discrete_sequence=["#1DB954"])
```



OVERVIEW:

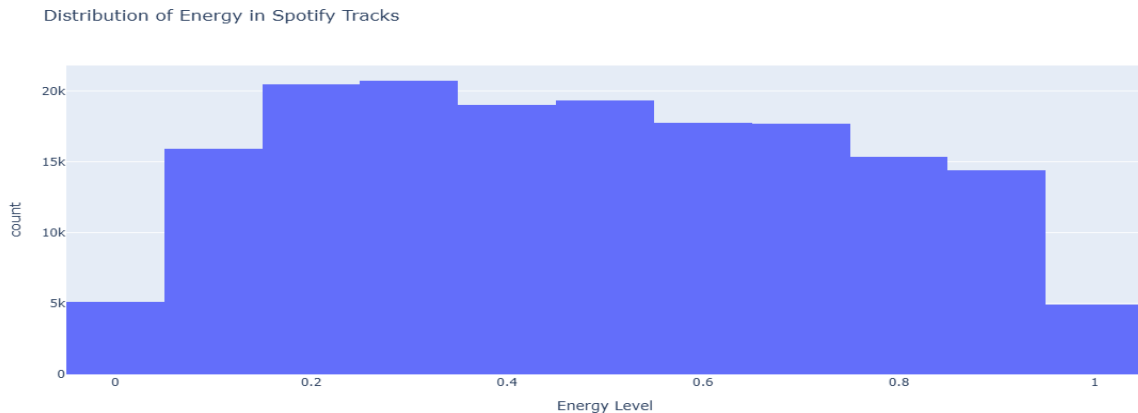
This Plotly histogram shows the distribution of danceability scores for Spotify tracks. The x-axis represents the danceability score (0 to 1), while the y-axis shows the number of songs falling within each score Range.

OUTCOME:

Most songs are concentrated in the mid-range danceability values (around 0.4 to 0.7), with a peak near 0.6. Very few tracks have extremely low or extremely high danceability. This indicates that Spotify's music library—and listener preference—leans toward moderately to highly danceable tracks, reflecting a general preference for rhythmically engaging music.

(B) Distribution of Energy in Spotify Tracks

```
# Task 4: Analyze the distribution of various features (e.g., danceability, energy, tempo).  
import plotly.express as px  
px.histogram(  
    df_tracks,  
    nbins = 20,  
    x="energy",  
    title="Distribution of Energy in Spotify Tracks",  
    labels={"energy": "Energy Level"})
```



OVERVIEW:

This Plotly histogram illustrates the distribution of energy levels across Spotify tracks. The x-axis represents the energy score (ranging from 0 to 1), while the y-axis shows the count of songs within each energy range.

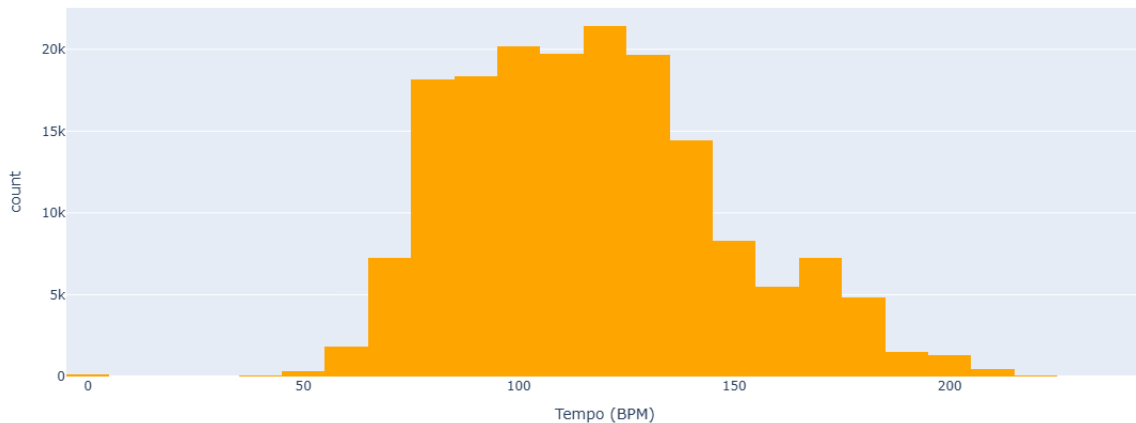
OUTCOME:

The distribution shows that a large number of tracks fall within the moderate to high energy range (approximately 0.4 to 0.8). Very low-energy tracks are relatively fewer, indicating that Spotify's catalog—and listener preference—favors lively, dynamic, and energetic music.

(C) Distribution of Tempo in Spotify Tracks

```
# Task 4: Analyze the distribution of various features (e.g., danceability, energy, tempo).
import plotly.express as px
px.histogram(
    df_tracks,
    x="tempo",
    nbins=40,
    title="Distribution of Tempo in Spotify Tracks",
    labels={"tempo": "Tempo (BPM)"},
    color_discrete_sequence=["orange"])
```

Distribution of Tempo in Spotify Tracks



OVERVIEW:

This Plotly histogram represents the distribution of tempo (measured in BPM – Beats Per Minute) across Spotify tracks. The x-axis shows the tempo values, while the y-axis indicates the number of songs within each tempo range.

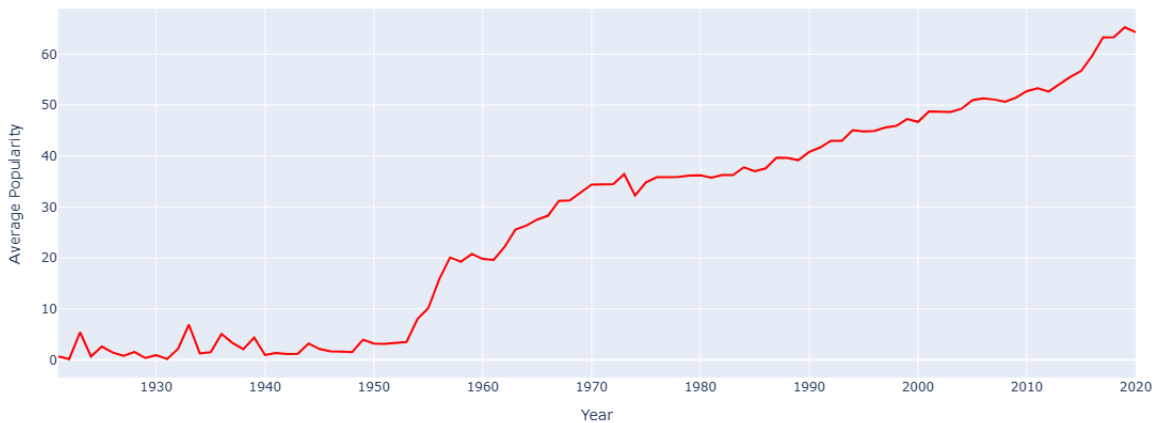
OUTCOME:

The majority of songs are concentrated between 90 and 140 BPM, with a noticeable peak around 110–120 BPM. Very slow and very fast tempo tracks are relatively few. This suggests that Spotify tracks—and listener preferences—favor moderate tempos, which are commonly found in pop, dance, and mainstream music.

(A) OVERALL SPOTIFY SONG POPULARITY TREND OVER

```
# Task 5 : Identify trends over time (e.g., popularity of genres, changes in song features).
import plotly.express as px
px.line(
    df_year,
    x="year",
    y="popularity",
    title="Overall Spotify Song Popularity Trend Over Time",
    labels={"year": "Year", "popularity": "Average Popularity"},
    color_discrete_sequence=["red"])
```

Overall Spotify Song Popularity Trend Over Time



OVERVIEW:

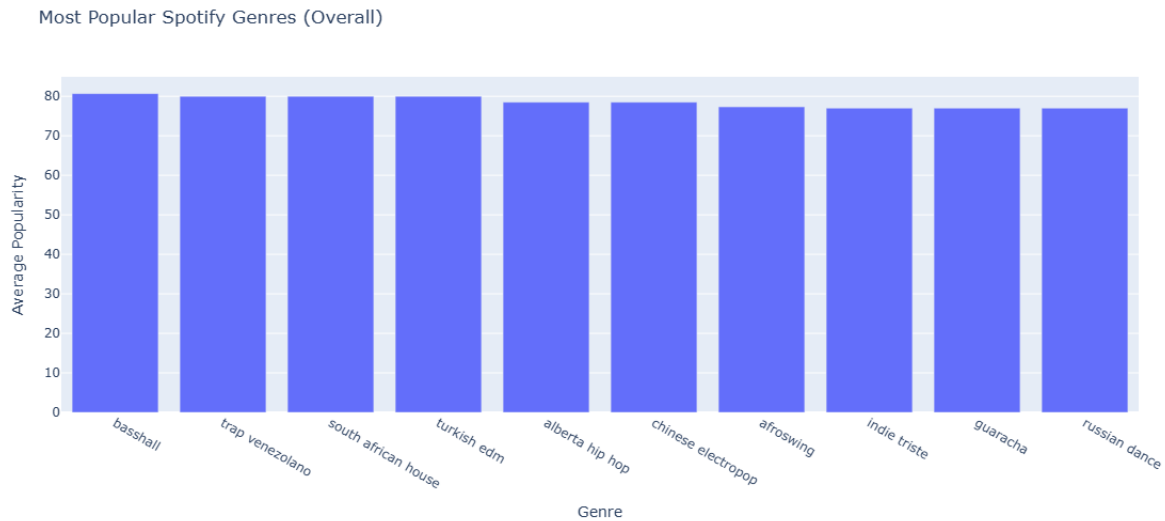
This Plotly line chart displays the overall trend of average song popularity on Spotify over time. The x-axis represents the year of release, and the y-axis shows the average popularity score of songs released in each year.

OUTCOME:

The graph shows a steady increase in song popularity over the years, with a significant rise starting from the 1950s onwards. Popularity grows more rapidly after the 1990s, reaching its highest levels in recent years. This trend indicates that modern songs tend to achieve higher popularity, likely due to digital streaming growth, wider audience reach, and changes in music consumption patterns.

(B) Most Popular Spotify Genres (Overall)

```
# Task 5 : Identify trends over time (e.g., popularity of genres, changes in song features).
import plotly.express as px
px.bar(
    top_genres,
    x="genres",
    y="popularity",
    title="Most Popular Spotify Genres (Overall)",
    labels={"genres": "Genre", "popularity": "Average Popularity"})
```



OVERVIEW:

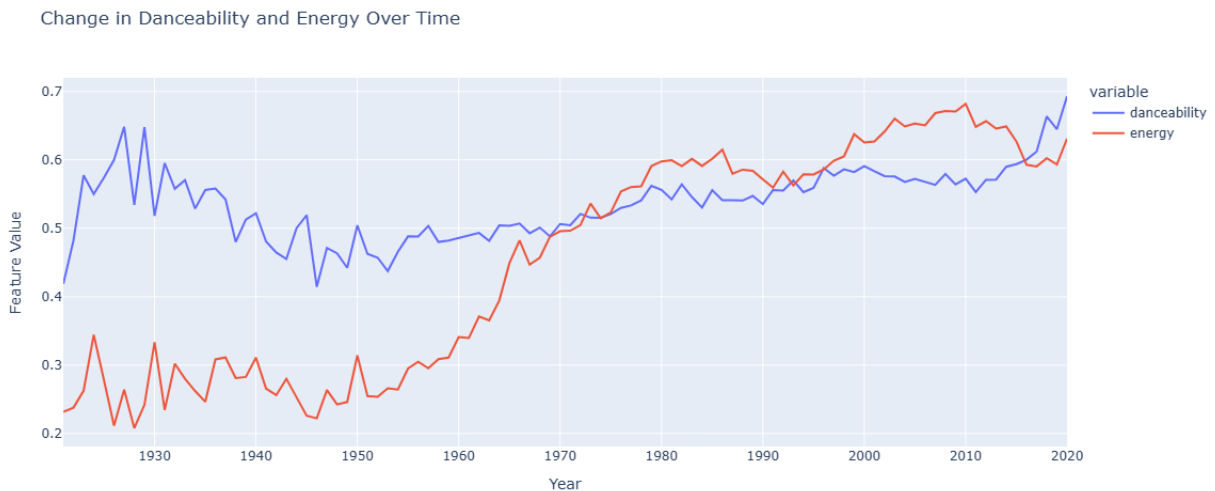
This Plotly bar chart presents the most popular Spotify genres overall, based on their average popularity scores. The x-axis lists different music genres, while the y-axis shows the average popularity of tracks belonging to each genre.

OUTCOME:

The chart shows that genres such as Basshall, Trap Venezolano, South African House, and Turkish EDM have the highest average popularity scores, all clustered in a similar range. This indicates that multiple global and regional genres perform strongly on Spotify, reflecting diverse listener preferences rather than dominance by a single genre.

(C) Change in Danceability and Energy Over Time

```
# Task 5 : Identify trends over time (e.g., popularity of genres, changes in song features)
import plotly.express as px
px.line(
    df_year,
    x="year",
    y=["danceability", "energy"],
    title="Change in Danceability and Energy Over Time",
    labels={"value": "Feature Value", "year": "Year"})
```



OVERVIEW:

This Plotly line chart shows the change in average danceability and energy of Spotify tracks over time. The x-axis represents the year of release, while the y-axis shows the average feature values for danceability (blue line) and energy (red line).

OUTCOME:

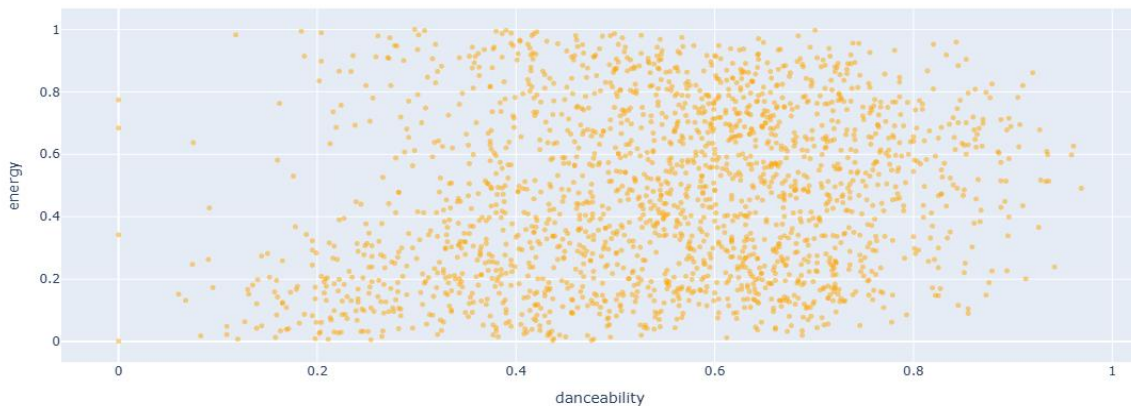
The graph reveals that both danceability and energy increase over time, especially after the 1960s. Energy shows a stronger upward trend, overtaking danceability in later years, indicating that modern music has become more energetic and dynamic. Danceability remains relatively stable with a gradual rise, suggesting that while rhythm remains important, higher energy has become a defining characteristic of contemporary music.

ENERGY VS DANCEABILITY (SAMPLED DATA)

```
# Task 6 : Examine correlations between different features (e.g., energy vs. danceability).
import plotly.express as px
sample_df = df_tracks.sample(2000, random_state=42)
fig = px.scatter(
    sample_df,
    x="danceability",
    y="energy",
    title="Energy vs Danceability (Sampled Data)",
    opacity=0.5)

fig.update_traces(marker_color="orange", marker_size=5)
```

Energy vs Danceability (Sampled Data)



OVERVIEW:

This Plotly scatter plot visualizes the relationship between danceability and energy for a sampled set of Spotify tracks. The x-axis represents danceability, while the y-axis represents energy, with each point corresponding to a song.

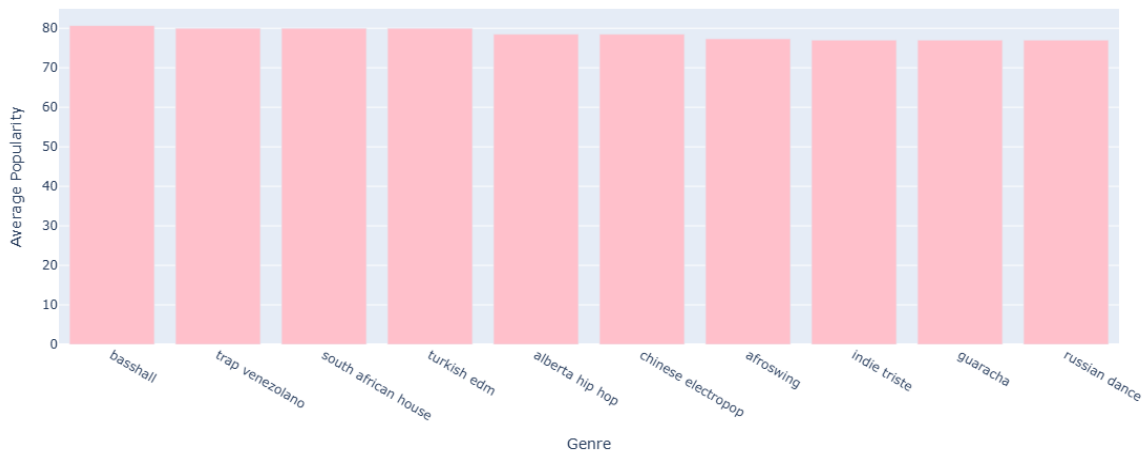
OUTCOME:

The plot shows a positive correlation between danceability and energy—songs with higher danceability generally tend to have higher energy. However, the wide spread of points indicates that this relationship is not perfectly linear, meaning there are energetic songs that are less danceable and vice versa. This highlights the diversity in musical styles and compositions within Spotify's catalog.

(A) USER PREFERRED GENRES ON SPOTIFY

```
# Task 7 : Analyze user preferences and listening habits.
import plotly.express as px
fig = px.bar(
    top_genres,
    x="genres",
    y="popularity",
    title="User Preferred Genres on Spotify",
    labels={"genres": "Genre", "popularity": "Average Popularity"})
fig.update_traces(marker_color="pink")
```

User Preferred Genres on Spotify



OVERVIEW:

This Plotly bar chart illustrates user-preferred music genres on Spotify based on the average popularity scores of tracks within each genre. The x-axis represents different genres, while the y-axis shows their average popularity.

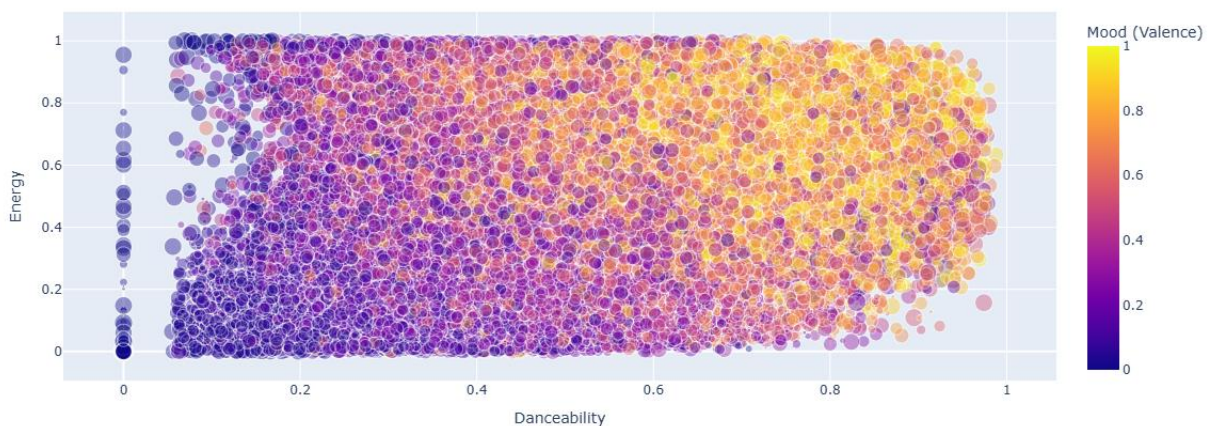
OUTCOME:

The chart shows that all listed genres have similarly high average popularity scores, with only minor variations between them. This indicates that Spotify users have diverse musical preferences, and no single genre overwhelmingly dominates user interest. Instead, listeners engage with a wide range of global and regional genres.

(B) User Preferences and Listening Habits (Single View)

```
# Task 7 : Analyze user preferences and listening habits.
import plotly.express as px
px.scatter(
    df_tracks,
    x="danceability",
    y="energy",
    size="popularity",
    color="valence",
    title="User Preferences and Listening Habits (Single View)",
    labels={
        "danceability": "Danceability",
        "energy": "Energy",
        "popularity": "Popularity",
        "valence": "Mood (Valence)"},
    opacity=0.4)
```

User Preferences and Listening Habits (Single View)



OVERVIEW:

This Plotly bubble scatter plot presents user preferences and listening habits in a single view. The x-axis represents danceability, the y-axis represents energy, and the color scale (valence) indicates the mood of the song (from sad/dark to happy/positive). Each bubble represents a track, combining multiple audio features into one visualization.

OUTCOME:

The plot shows that most users prefer songs with moderate to high danceability and energy, often associated with positive moods (higher valence). Tracks with higher danceability generally trend toward higher energy and happier moods, indicating that listeners favor upbeat, energetic, and mood-lifting music. The wide spread also highlights diverse listening habits, with users engaging across different energy and mood levels.

(A) TOP 10 SPOTIFY GENRES BY AVERAGE POPULARITY

Task 8 : Create visualizations to represent key findings (e.g., bar charts, line graphs, scatter plots).

```
import plotly.express as px
```

```
px.bar(
```

```
    top_genres,
```

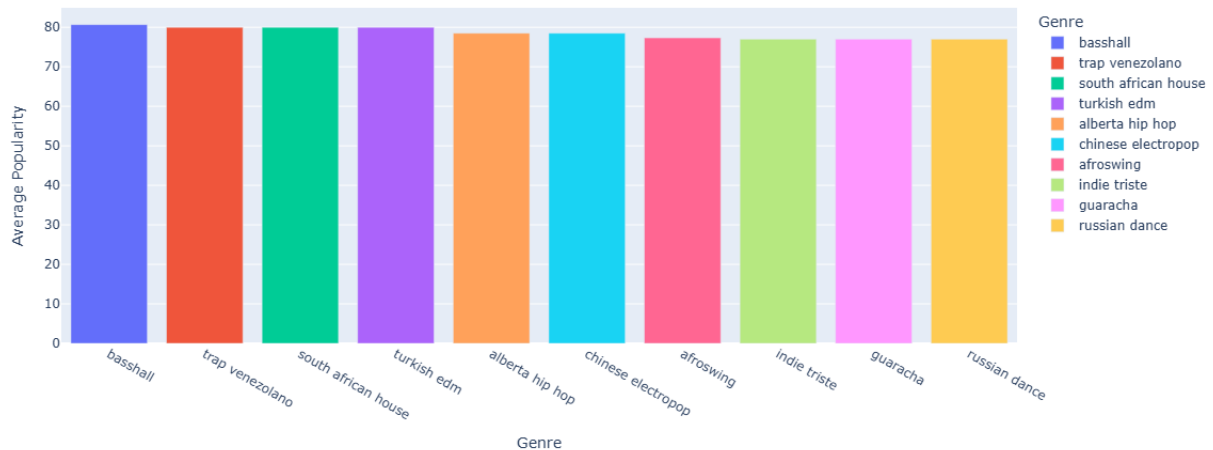
```
    x="genres",
```

```
    y="popularity",
```

```
    title="Top 10 Spotify Genres by Average Popularity",
```

```
    labels={"genres": "Genre", "popularity": "Average Popularity"},color = "genres")
```

Top 10 Spotify Genres by Average Popularity



OVERVIEW:

This Plotly bar chart displays the Top 10 Spotify genres based on average popularity. The x-axis lists the genres, while the y-axis represents their average popularity scores. Each bar highlights how well a genre performs in terms of listener popularity.

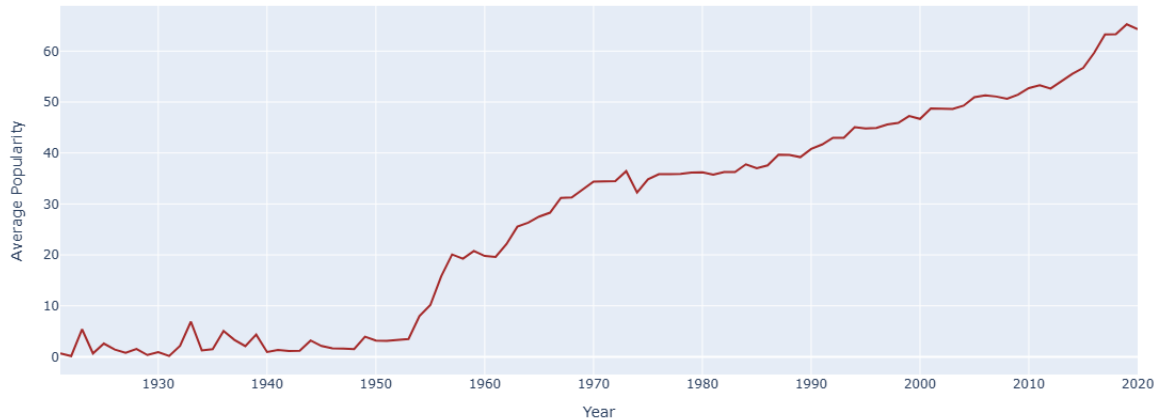
OUTCOME:

The chart shows that all top 10 genres have very similar and high popularity scores, with Basshall, Trap Venezolano, South African House, and Turkish EDM slightly leading. This suggests that multiple genres perform equally well on Spotify, reflecting diverse global listening preferences rather than dominance by a single genre.

(B) SPOTIFY SONG POPULARITY TREND OVER TIME

```
# Task 8 : Create visualizations to represent key findings (e.g., bar charts, line graphs, scatter plots).
import plotly.express as px
px.line(
    df_year,
    x="year",
    y="popularity",
    title="Spotify Song Popularity Trend Over Time",
    labels={"year": "Year", "popularity": "Average Popularity"},
    color_discrete_sequence=["brown"])
```

Spotify Song Popularity Trend Over Time



OVERVIEW:

This Plotly line graph illustrates the trend of average Spotify song popularity over time. The x-axis represents the year, and the y-axis shows the average popularity score of songs released in each year.

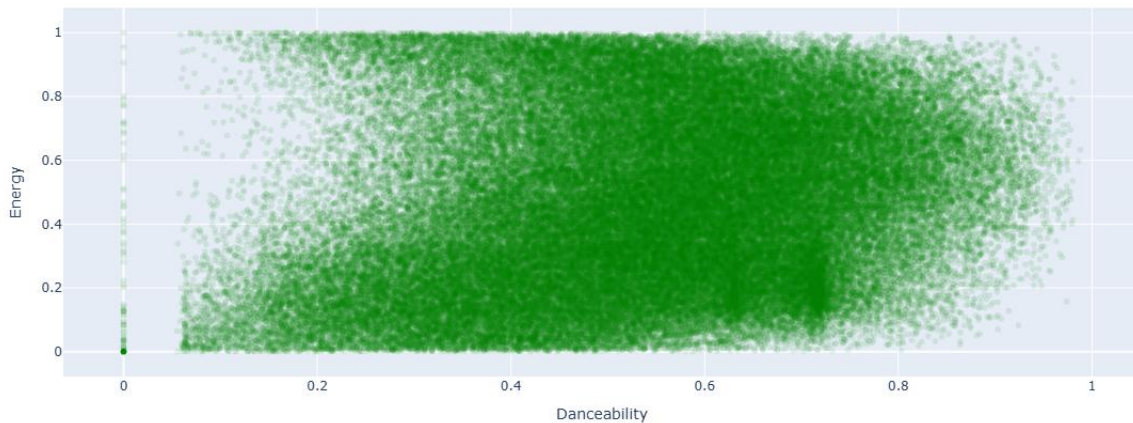
OUTCOME:

The graph shows a clear upward trend in song popularity, with a noticeable increase beginning around the 1950s and continuing steadily into recent years. The sharp rise in later decades suggests that modern songs tend to achieve higher popularity, likely driven by digital streaming platforms, global reach, and evolving listener engagement.

(C) ENERGY VS DANCEABILITY OF SPOTIFY TRACKS

```
# Task 8 : Create visualizations to represent key findings (e.g., bar charts, line graphs, scatter plots).
import plotly.express as px
px.scatter(
    df_tracks,
    x="danceability",
    y="energy",
    title="Energy vs Danceability of Spotify Tracks",
    labels={"danceability": "Danceability", "energy": "Energy"},
    opacity=0.1,    color_discrete_sequence=["green"])
```

Energy vs Danceability of Spotify Tracks



OVERVIEW:

This Plotly scatter plot visualizes the relationship between danceability and energy for a large set of Spotify tracks. The x-axis represents danceability, while the y-axis represents energy, with each point corresponding to a song.

OUTCOME:

The plot shows a general positive relationship between danceability and energy—songs with higher danceability tend to also have higher energy. However, the dense clustering and wide spread of points indicate significant variation, meaning that not all energetic songs are highly danceable and vice versa. This highlights the diversity of musical characteristics across Spotify tracks.

(A) CORRELATION HEATMAP OF SPOTIFY AUDIO

```
# Task 9 | Use advanced visualization techniques (e.g., heatmaps, pair plots) to uncover deeper insights.
import plotly.express as px
px.imshow(
    corr_matrix,
    text_auto=True,
    color_continuous_scale="RdBu",
    title="Correlation Heatmap of Spotify Audio Features")
```

Correlation Heatmap of Spotify Audio Features



OVERVIEW:

This Plotly heatmap displays the correlation between key Spotify audio features such as danceability, energy, tempo, valence, loudness, acousticness, and speechiness. The color scale represents the strength and direction of correlation, ranging from strong negative to strong positive relationships.

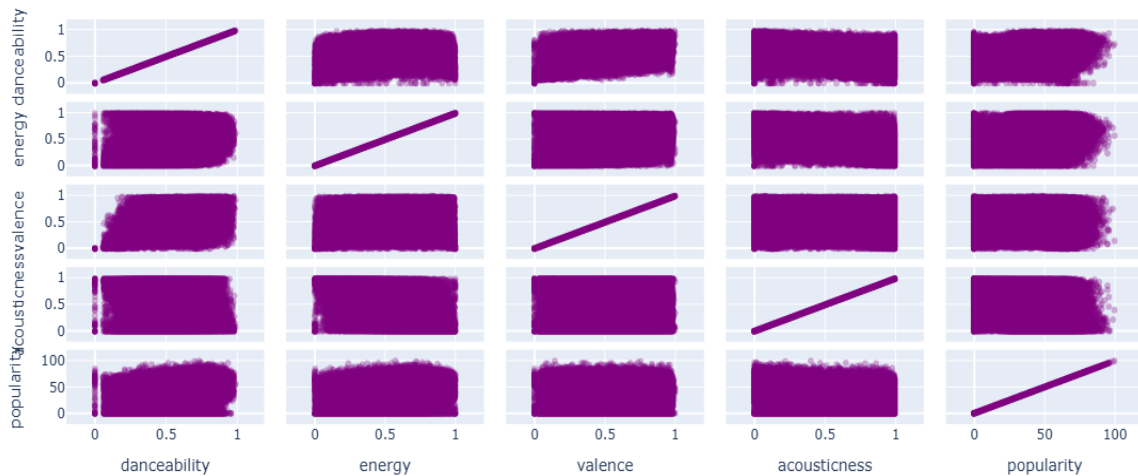
OUTCOME:

The heatmap shows a strong positive correlation between energy and loudness, indicating that louder songs are generally more energetic. There is a moderate positive relationship between danceability and valence, suggesting happier songs tend to be more danceable. Acousticness is negatively correlated with energy and loudness, meaning acoustic tracks are typically softer and less energetic. Overall, the heatmap highlights how different audio features interact and influence the overall musical characteristics of Spotify tracks.

(B) PAIR PLOT OF SPOTIFY AUDIO FEATURES

```
# Task 9 : Use advanced visualization techniques (e.g., heatmaps, pair plots) to uncover deeper insights.
import plotly.express as px
px.scatter_matrix(
    df_tracks,
    dimensions=pair_features,
    title="Pair Plot of Spotify Audio Features",
    opacity=0.3,color_discrete_sequence=["purple"])
```

Pair Plot of Spotify Audio Features



OVERVIEW:

This Plotly pair plot visualizes the pairwise relationships between multiple Spotify audio features—including danceability, energy, valence, acousticness, and popularity. Each subplot shows how two features relate to each other, helping compare patterns across all combinations in a single view.

OUTCOME:

The pair plot reveals that energy and danceability show a generally positive relationship, while acousticness tends to have a negative relationship with energy and popularity. Popular songs are more commonly associated with higher energy and danceability, whereas highly acoustic tracks are less frequently popular. Overall, the plot highlights how audio features collectively influence song popularity rather than any single feature acting alone.

CONCLUSION

- The project focused on analyzing Spotify music data to understand patterns in song popularity, audio features, genres, and listener preferences using Exploratory Data Analysis (EDA).
- Analysis of danceability and energy distributions showed that most popular tracks fall within moderate ranges, indicating listener preference for balanced and engaging music rather than extremes.
- Tempo (BPM) analysis revealed that the majority of songs cluster around mid-range tempos, suggesting that steady rhythms dominate mainstream music.
- The song popularity trend over time showed a consistent upward movement, reflecting the growth of digital streaming platforms, improved accessibility, and algorithm-driven music discovery.
- Genre-based analysis highlighted that both mainstream and niche genres can achieve high popularity, emphasizing Spotify's support for diverse musical tastes.
- User preference visualizations confirmed that listeners have varied and personalized tastes, reinforcing the importance of recommendation systems.
- The relationship between danceability, energy, and valence indicated that energetic and dance-friendly songs often have a more positive mood, making them suitable for playlists like workout and party mixes.
- Scatter plots showed no single audio feature determines popularity, proving that song success depends on a combination of multiple musical attributes.
- Correlation analysis identified strong relationships such as:
 - . Positive correlation between energy and loudness
 - . Negative correlation between acousticness and energy

These relationships align with real-world music production patterns.

- Pair plots further validated feature interactions and demonstrated how audio features collectively influence popularity.
- Overall, the project successfully applied data analytics and basic data science techniques to extract meaningful insights from large-scale music data.

The EDA provides a strong foundation for future work such as:

- Song popularity prediction.

- Music recommendation systems.

- Trend forecasting and personalization models.

ACKNOWLEDGEMENT

I wish to express my heartfelt thanks to everyone involved in my EDA project.

First and foremost, I am grateful to Mr. Tushar Bhadouria sir for providing the necessary resources and guidance for undertaking this project. Their patience and encouragement made the entire process enjoyable.

I would also like to thank for advocating technology education at our institution. The computer lab facilities provided me with the opportunity to learn new skills through hands-on practice.

Finally, I appreciate the constructive feedback and suggestions from my classmates, which played a crucial role in refining my project at various stages.

The successful completion of this project was a collective effort, and I am deeply appreciative of the time and care invested by everyone. This learning experience has been incredibly fruitful, and I will always remember the collaborative efforts that made it possible.

THANK YOU

<https://colab.research.google.com/drive/1-sSwK9iL5dnpz5dJl-uUwyC0i8ABECp6?usp=sharing>